



كلية الهندسة المعلوماتية

جامعة دمشق

السنة الرابعة

قسم الذكاء الصناعي واللغات الطبيعية

مشروع مادة خوارزميات البحث

لعبة Senet

إعداد :

شام زياد العلي (43176)

حيدر حسين ابراهيم (43120)

جودي خليل برادعي (43092)

آيه يزن الحصري (43052)

سدره وليد الحمد (43161)

إشراف المهندسة :

هيام نجم الدين.

المحتويات

1. [مقدمة عن اللعبة](#)
2. [تمثيل المسألة كمسألة بحث](#)
3. [البنية البرمجية للمشروع](#)
4. [نماذج البيانات \(Data Models\)](#)
5. [منطق اللعبة والقواعد](#)
6. [خوارزمية Expectiminimax](#)
7. [واجهة المستخدم والتحكم](#)
8. [احتمالات رمي العصي](#)
9. [كيف تم التفكير وطباعة بعض النتائج](#)
10. [التقسيم على أفراد المجموعة](#)

مقدمة

1. مقدمة عن اللعبة

1.1 نبذة تاريخية

لعبة السينت (**Senet**) هي واحدة من أقدم ألعاب الطاولة المعروفة في التاريخ، تعود إلى مصر القديمة. كلمة "سينت" تعني في اللغة المصرية القديمة "العبور"، في إشارة إلى العبور نحو العالم الآخر. عُثر على ألواح السينت داخل المقابر الفرعونية، وتعود أقدم الألواح المعروفة إلى عصر الدولة الوسطى (نحو 1710-2050 قبل الميلاد).

2.1 قواعد اللعبة المستخدمة

القواعد الأصلية للعبة السينت غير معروفة بشكل مباشر، لكننا استخدمنا في هذا المشروع قواعد كِنْدَال (**Kendall's Rules**) التي أعيد بناؤها بناءً على الصور المنقوشة والقطع الأثرية.

3.1 مكونات اللعبة

- الرقعة 30: مربعاً مرتبة في 3 صفوف 10×10 أعمدة
- الأحجار 7: أحجار لكل لاعب (أبيض وأسود)
- العصي 4: عصي خشبية لتحديد عدد الحركات (كل عصا لها وجه فاتح ووجه داكن)

تمثيل-المسألة

2. تمثيل المسألة كمسألة بحث

1.2 - الحالة (State)

الحالة في لعبة السينت تتكون من:

- موقع جميع الأحجار على الرقعة (14 حجراً)
- اللاعب الحالي الذي له الدور
- قيمة رمية العصي (إن وجدت)
- حالة كل حجر : على اللوح، منتهي.

2.2 تابع الانتقال (Transition Function)

الانتقال من حالة S إلى حالة S' يتم عبر:

1. رمي العصي عقدة حظ (Chance Node)
2. اختيار حركة من الحركات الممكنة
3. تطبيق قواعد المربعات الخاصة

3.2 الكلفة (Cost)

هي المسافة التي يتحرك بها الحجر وإذا رجع الحجر للوراء بسبب التبديل يتم إنقاص نفس عدد الخطوات رجع بها للوراء .

4.2 حالة البداية (Initial State)

- 14 حجراً موزعة بالتناوب على المربعات 1-14
- 7 أحجار بيضاء (للكمبيوتر) في المربعات الفردية: 1, 3, 5, 7, 9, 11, 13
- 7 أحجار سوداء (للإنسان) في المربعات الزوجية: 2, 4, 6, 8, 10, 12, 14

5.2 حالة النهاية (Goal State)

تنتهي اللعبة عندما ينجح أحد اللاعبين في إخراج جميع أحجاره السبعة من الرقعة.

البنية البرمجية:

3. البنية البرمجية للمشروع

تم تصميم المشروع باستخدام مبادئ البرمجة كائنية التوجه (OOP) بشكل احترافي:

:Data Models 1.3

:class Cell.1

الوصف العام:

يمثل هذا الكلاس مربعاً واحداً (خلية) من مربعات لوح اللعبة البالغ عددها 30 مربعاً. كل خلية لها رقم تعريف (index) يحدد موقعها على اللوح، ونوع (type) يحدد ما إذا كانت خلية عادية أو خلية خاصة ذات قواعد معينة، بالإضافة إلى إمكانية احتوائها على حجر (Piece) من أحجار اللاعبين. هذا الكلاس يشكل الوحدة الأساسية لتمثيل الرقعة، حيث أن لوح اللعبة يتكون من 30 كائن من نوع Cell، وهو ضروري لتطبيق قواعد اللعبة الخاصة بالبيوت المختلفة.

المتغيرات الرئيسية:

index: رقم المربع على اللوح (من 1 إلى 30) وهو ثابت لا يتغير
type: نوع المربع (عادي أو أحد البيوت الخاصة مثل بيت البعث، بيت السعادة، إلخ)
occupant: الحجر الموجود حالياً على هذا المربع (يكون null إذا كان المربع فارغاً)

البناءات (Constructors):

الباني الأول: يأخذ رقم المربع ويحدد نوعه تلقائياً باستخدام الدالة CellType.getTypeForPosition() ويبدأ المربع فارغاً

الباني الثاني: بناء نسخ (Copy Constructor) ينشئ نسخة من خلية موجودة، مفيد عند إنشاء نسخ من حالات اللعبة أثناء تطبيق خوارزمية Expectiminimax

الدوال الرئيسية:

getters: دوال للحصول على معلومات المربع (الرقم، النوع، الحجر الموجود)
setOccupant(): لوضع أو إزالة حجر من المربع
isEmpty(): للتحقق من أن المربع فارغ
isOccupiedBy(): للتحقق من أن المربع مشغول بحجر يعود للاعب معين
isSpecialCell(): للتحقق من أن المربع ذو قواعد خاصة
getCSymbol(): تُرجع رمزاً نصياً يمثل المربع عند الطباعة، حيث تستخدم رموز خاصة للبيوت الخاصة (Ψ للبعث، ☺ للسعادة، ≈ للماء، إلخ) أو رمز الحجر إن كان موجوداً

2. class CellType :

الوصف العام:

هو Enumeration (تعداد) يحدد جميع أنواع المربعات الممكنة في لعبة Senet. يستخدم هذا التعداد لتصنيف المربعات وتطبيق القواعد الخاصة بكل نوع وفقاً لقواعد Kendall المذكورة في المشروع هذا الكلاس يسهل تطبيق القواعد الخاصة بكل بيت دون الحاجة للتحقق من أرقام المربعات بشكل يدوي، مما يجعل الكود أكثر وضوحاً وسهولة في الصيانة والتعديل.

القيم المعرفة:

NORMAL: المربعات العادية (ليس لها قواعد خاصة)
REBIRTH: بيت البعث (المربع 15) - يُعاد إرسال الأحجار إليه
HAPPINESS: بيت السعادة (المربع 26) - يجب الهبوط عليه مباشرة
WATER: بيت الماء (المربع 27) - يعيد الحجر إلى بيت البعث فوراً
THREE_TRUTHS: بيت الحقائق الثلاث (المربع 28) - يحتاج رمية 3 للخروج
RE_ATOUM: بيت إعادة أتوم (المربع 29) - يحتاج رمية 2 للخروج
HORUS: بيت حورس (المربع 30) - يمكن الخروج بأي رمية

المتغيرات:

position: رقم المربع على اللوح

englishName: الاسم الإنجليزي للبيت

الدوال:

get TypeForPosition(): دالة ثابتة (static) مهمة جداً تأخذ رقم مربع وتُرجع نوعه، تُستخدم عند إنشاء الخلايا لتحديد ما إذا كان المربع عادياً أم خاصاً

3. class Move :

الوصف العام:

يمثل هذا الكلاس حركة واحدة في اللعبة، أي تحريك حجر معين من موقع إلى موقع آخر. هو جزء أساسي من تمثيل المسألة كمسألة بحث، حيث يمثل "تابع الانتقال" (transition function) الذي ينقلنا من حالة (state) إلى حالة أخرى.

كائنات **Move** تمثل الانتقالات الممكنة من حالة لعبة إلى أخرى. في خوارزمية **Expectiminimax**، نقوم بتوليد جميع الحركات الممكنة لحالة معينة، ثم نطبق كل حركة لنحصل على حالات جديدة يتم تقييمها. هذا الكلاس يسهل تتبع الحركات المختارة وطباعة معلومات تنفيذ الخوارزمية كما هو مطلوب في المشروع.

المتغيرات الرئيسية:

piece: الحجر الذي سيتم تحريكه

fromPosition: الموقع الأصلي للحجر

toPosition: الموقع الهدف للحجر

steps: عدد المربعات التي سيتحركها الحجر (يساوي قيمة رمية العصي)

isSwap: هل ستحدث مبادلة مع حجر الخصم؟

isExit: هل ستؤدي هذه الحركة لخروج الحجر من اللوح؟

البناءات:

الباني الأول: بناء مبسط يأخذ الحجر وعدد الخطوات، ويحسب باقي المعلومات تلقائياً

الباني الثاني: بناء كامل يسمح بتحديد جميع التفاصيل، مفيد عند توليد الحركات المعقدة

الدوال:

دوال **getters**: للحصول على تفاصيل الحركة

toString(): تمثيل نصي مختصر للحركة

getDetailedDescription(): وصف تفصيلي للحركة يُستخدم عند طباعة معلومات الخوارزمية

equals() و **hashCode()**: للمقارنة بين الحركات، مهم عند التحقق من الحركات المكررة

4. class NodeType

الوصف العام:

هو **Enumeration** يحدد أنواع العقد الثلاثة في شجرة البحث الخاصة بخوارزمية **Expectiminimax**. هذه الخوارزمية تختلف عن **Minimax** العادية بوجود عقد الحظ (**Chance nodes**) التي تمثل رميات العصي العشوائية.

القيم المعرفة:

MAX: عقدة تعظيم القيمة

MIN: عقدة تصغير القيمة

CHANCE: عقدة الحظ، تمثل رمي العصي (خطوة عشوائية تحدد عدد الخطوات المتاحة)

المتغير:

arabicName: اسم نوع العقدة (مخزن بالإنجليزية في الكود الحالي لكن يمكن تعديله للعربية)

الأهمية :

في خوارزمية Expectiminimax، يتناوب نوع العقدة بين MAX/MIN/CHANCE:

عقدة MAX: نختار الحركة التي تعطي أقصى قيمة

عقدة MIN: نفترض أن الخصم يختار الحركة التي تعطي أدنى قيمة

عقدة CHANCE: نحسب القيمة المتوقعة (expected value) بضرب قيمة كل رمية باحتمالها ثم جمع النتائج

هذا التمييز بين أنواع العقد ضروري لتطبيق المعالجة الصحيحة في كل مستوى من شجرة البحث، وأيضاً لطباعة معلومات تنفيذ الخوارزمية بشكل واضح كما هو مطلوب في المشروع.

Class Piece. 5

الوصف العام:

يمثل هذا الكلاس حجراً واحداً من أحجار اللاعب. كل حجر له معرف فريد (id) ، ينتمي إلى لاعب معين (owner)، وله موقع على لوح اللعبة (position) ، وحالة تحدد ما إذا كان على الرقعة أو قد انتهى (state). هذا الكلاس يمثل وحدة تحكم أساسية في اللعبة، حيث يتم تتبع كل حجر بشكل منفصل ومراقبة حالته أثناء سير اللعبة.

المتغيرات الرئيسية:

- id : معرف فريد للحجر (رقم من 0-7 لكل لاعب) - ثابت لا يتغير
- owner : اللاعب التي ينتمي إليه الحجر (WHITE أو BLACK)
- position : الموقع الحالي للحجر على اللوح (من 1 إلى 30، أو 0 إذا انتهى)
- state : حالة الحجر (ON_BOARD أو FINISHED)
- needsExactRoll : تُستخدم للبيوت التي تحتاج رمية محددة للخروج (مثل بيت الحقائق الثلاث يحتاج 3)

البناءات:

- الباني الأول : يأخذ معرف الحجر وموقعه الأولي، ويضع الحالة الافتراضية ON_BOARD

- الباني الثاني :بناء نسخ (Copy Constructor) ينسخ جميع معلومات الحجر ، مهم عند نسخ حالات اللعبة للخوارزمية

الدوال الرئيسية:

- **getters** :للحصول على معلومات الحجر (المعرّف، المالك، الموقع، الحالة، الرمية المطلوبة)
 - **setters**:تعديل موقع الحجر أو حالته أو الرمية المطلوبة
 - **isFinished()** : للتحقق إذا كان الحجر قد انتهى من اللعبة
 - **isOnBoard()** : للتحقق إذا كان الحجر على الرقعة حالياً
 - **toString()** : تمثيل نصي مختصر للحجر (مثل "W1" أو "B3")
 - **getDetailedInfo()** : وصف تفصيلي يشمل موقع الحجر والرمية المطلوبة إن وجدت
-

6 . Class PieceStat :

الوصف العام:

هو Enumeration يحدد الحالات الممكنة لأي حجر في اللعبة. كل حجر يكون إما على الرقعة أو قد انتهى من اللعبة. هذا التعداد يُسهّل التحقق من حالة الحجر ويجعل الكود أوضح وأكثر سهولة في الصيانة بدلاً من استخدام أرقام أو نصوص غير منظمة.

القيم المعرفة:

- **ON_BOARD** : الحجر موجود على رقعة اللعبة (يمكن تحريكه)
- **FINISHED** : الحجر قد انتهى من اللعبة (وصل للمربع 30 أو ما بعده)

المتغيرات:

- **arabicName** : نص وصفي يمثل الحالة ("on the board") أو ("out of the game")

الدوال:

- **getArabicName()** : تُرجع الوصف النصي للحالة

الأهمية:

هذا التعداد ضروري لتحديد اللاعب أي حجر يمكنه تحريكه وأي حجر قد انتهى. كما يُستخدم في قواعد اللعبة لتحديد ما إذا كانت حركة معينة صحيحة أم لا.

7 . Class PlayerType :

الوصف العام:

هو Enumeration يحدد نوع اللاعب في اللعبة. اللعبة تحتوي على لاعبين فقط

- WHITE :الكمبيوتر (AI)

- BLACK (الإنسان/اللاعب الحقيقي).

هذا التعداد يستخدم في تحديد من يمتلك كل حجر، ومن يجب أن يلعب في كل دور.

القيم المعرفة:

• WHITE :اللاعب الكمبيوتر يُمثل برمز("W")

• BLACK :اللاعب الإنسان يُمثل برمز("B")

المتغيرات:

• Symbol :رمز نصي يمثل اللاعب عند الطباعة ("W" أو "B")

الدوال:

• `getSymbol()`: تُرجع رمز اللاعب لاستخدامه في الطباعة والعروض البصرية

• `getOpponent()`: تُرجع اللاعب الخصم (إذا كنت WHITE ترجع BLACK والعكس)

الأهمية:

هذا التعداد يُسهل التعامل مع اللاعبين بطريقة آمنة من الناحية النوعية (Type-safe) ، ويوفر طريقة سهلة للتبديل بين اللاعب والخصم دون الحاجة للتحقق اليدوي من الأسماء. كما أنه يُستخدم في تحديد المالك الصحيح للأحجار وفي خوارزمية البحث لتحديد نوع العقدة MAX للـ WHITE و MIN للـ BLACK .

:AI 2.3

: Class SenetAI

الوصف العام:

يمثل هذا الكلاس الذكاء الاصطناعي الذي يلعب ضد اللاعب البشري. يستخدم خوارزمية Expectiminimax لاختيار أفضل حركة ممكنة.

Expectiminimax هي نسخة معدلة من Minimax تتعامل مع الألعاب ذات العنصر العشوائي (Stochastic Games).

الفكرة الأساسية:

- عقد MAX: اختر الحركة ذات القيمة الأعلى (الكمبيوتر يُعظم)

- عقد MIN: افترض أن الخصم يختار الحركة ذات القيمة الأدنى

- عقد: **CHANCE**: احسب القيمة المتوقعة لجميع الرميات الممكنة

المتغيرات الرئيسية:

maxDepth: أقصى عمق للبحث في شجرة الحركات (2-5 عادةً).

- عمق أكبر = ذكاء أقوى لكن أبطأ

- عمق أصغر = ذكاء أضعف لكن أسرع

nodesVisited: عدد لعدد العقد التي تم زيارتها أثناء البحث (لإحصائيات).

verboseMode: هل نطبع تفاصيل تنفيذ الخوارزمية؟

aiPlayer: اللاعب الذي يمثله الـ **AI** هو **(PlayerType.WHITE)**

الأوزان في دالة التقييم:

- **PROGRESS_WEIGHT = 2.0** وزن تقدم أحجار الـ **AI**

- **OPPONENT_PROGRESS_WEIGHT = 1.5** وزن تقدم أحجار الخصم

- **FINISHED_BONUS = 50.0** مكافأة لكل حجر منته

- **SPECIAL_CELL_BONUS = 10.0** مكافأة للأحجار في المنطقة الأخيرة

- **SAFETY_BONUS = 5.0** مكافأة للمواقع الآمنة

الدوال الرئيسية:

1. **getBestMove(GameState state)**: تُرجع أفضل حركة ممكنة للـ **AI** في حالة معينة.

2. **expectiminimax(GameState state, int depth, NodeType nodeType)**: الدالة العودية الرئيسية لخوارزمية **Expectiminimax**.

حالات التوقف:

```
if (depth == 0 || state.isGameOver()) {  
    return heuristic(state);  
}
```

التفرع حسب نوع العقدة:

- **CHANCE** → استدعاء **chanceNode()**

- **MAX** → استدعاء **maxNode()**

- **MIN** → استدعاء **minNode()**

3. `chanceNode(GameState state, int depth, boolean printDetails)` : معالجة عقدة الحظ - حساب القيمة المتوقعة (Expected Value).

4. `maxNode(GameState state, int depth, boolean printDetails)` : معالجة عقدة التعظيم - الـ AI يختار أفضل حركة له.

5. `minNode(GameState state, int depth, boolean printDetails)` : معالجة عقدة التصغير - نفترض أن الخصم يختار أسوأ حركة للـ AI.

الخوارزمية : مشابهة لـ `maxNode` لكن نستخدم `min` بدلاً من `max`.

6. `heuristic(GameState state)` : دالة التقييم - تُعطي قيمة رقمية لحالة معينة من وجهة نظر الـ AI .

مكونات التقييم:

(1) فحص انتهاء اللعبة:

```
if (state.isGameOver()) {  
    if (winner == aiPlayer)  
        return +10000.0;      فوز AI  
    else  
        return -10000.0;     فوز الخصم  
}
```

(2) التقدم الإجمالي:

```
score += PROGRESS_WEIGHT × aiProgress  
score -= OPPONENT_PROGRESS_WEIGHT × opponentProgress
```

حيث `calculateProgress()` = مجموع مواقع الأحجار + 35 لكل حجر منتهٍ

(3) مكافأة الأحجار المنتهية:

لكل حجر منتهٍ للـ AI:

```
score += 50.0
```

لكل حجر منتهٍ للخصم:

score -= 50.0

4) مكافأة الأحجار القريبة من النهاية:

إذا: position >= 26

score += 10.0 × (position - 25)

5) مكافأة المواقع الآمنة:

+5 نقاط للأحجار في مواقع آمنة

6) عقوبة المواقع الخطرة

20- نقطة للحجر في بيت الماء (27)

3.3 game_logic :

1. class Dice :

الوصف العام:

يمثل هذا الكلاس نظام رمي العصي (throwing sticks) في اللعبة، كانت تُستخدم 4 عصي خشبية، كل عصا لها وجهان: وجه فاتح (light) ووجه داكن (dark). يتم رمي العصي الأربعة معاً، وتُحسب النتيجة بناءً على عدد العصي التي أظهرت الوجه الداكن.

هذا الكلاس مسؤول عن تمثيل التوزيع الاحتمالي لنتائج الرمي، وهو جزء أساسي من خوارزمية Expectiminimax حيث يمثل "عقد الحظ" (Chance Nodes) "كل رمية محتملة لها احتمال محدد، وهذه الاحتمالات يتم استخدامها لحساب القيمة المتوقعة (expected value) في عقد الحظ.

القيم المعرفة:

- PROBABILITIES: هنا في hashmap تم ربط كل قيمة الحدث بقيمته الاحتمالية .

- POSSIBLE_ROLLS: جميع الأحداث الممكنة .

المتغيرات:

random (Random) : كائن static من نوع Random يُستخدم لتوليد نتائج عشوائية عند محاكاة رمي العصي.

الدوال:

1. `getProbability(int roll)`: دالة ثابتة تُرجع احتمال الحصول على نتيجة معينة
 2. `getPossibleRolls()`: تُرجع مصفوفة بجميع النتائج الممكنة {1, 2, 3, 4, 5}
 3. `getAllProbabilities()`: ترجع نسخة من جميع الاحتماليات.
 4. `printProbabilityTable()`: طباعة جدول الاحتماليات.
 5. `throwSticksWithVisualization()`: محاكاة مرئية لرمي العصي الأربعة، مع عرض نتيجة كل عصا والنتيجة النهائية
 6. `throwSticksWithVisualization()`: رمي العصي مع طباعتها على الكونسول.
- احتمالات رمي العصي

الحساب الرياضي

يتم رمي 4 عصي، كل واحدة لها وجه فاتح (0) ووجه داكن (1).
 باستخدام التوافق :

$$P(k \text{ أوجه الداكنة}) = C(n, k) / 16$$

حيث $C(4, k)$ التوافق.

جدول الاحتمالات:

عدد الأوجه الداكنة	معامل ذي الحدين	النتيجة	الاحتمال	النسبة
0	$C(4, 0) = 1$	5	1/16	6.25%
1	$C(4, 1) = 4$	1	4/16	25%
2	$C(4, 2) = 6$	2	6/16	37.5%
3	$C(4, 3) = 4$	3	4/16	25%
4	$C(4, 4) = 1$	5	1/16	6.25%

2. class Rules

الوصف العام:

يمثل هذا الكلاس القواعد الأساسي للعبة هو المسؤول عن تحديد صحة الحركات، تطبيق القواعد الخاصة بكل نوع من المربعات، وإدارة العقوبات المرتبطة بالبيوت الخاصة. يعتبر هذا الكلاس حلقة الوصل بين نموذج البيانات (Board, Piece, Move) وبين منطق اللعبة الفعلي.

1. `getValidMoves(Board board, PlayerType player, int diceValue)`: الدالة الأساسية لتوليد جميع الحركات الممكنة للاعب عند رمية عصي محددة.
2. `canMovePiece(Board board, Piece piece, int steps)`: يتحقق من إمكانية تحريك حجر معين بعدد خطوات محدد، وتطبق جميع قواعد اللعبة الخاصة بالبيوت الخاصة والحركات المسموحة.

القواعد المطبقة:

(a) قواعد البيوت الخاصة للخروج:

- **THREE_TRUTHS (المربع 28)**: الحجر محتجز ولا يمكنه الخروج إلا برمية 3 بالضبط. إذا حصل اللاعب على 3، يخرج الحجر مباشرة (`isExit = true`)
- **RE_ATOUM (المربع 29)**: الحجر محتجز ولا يمكنه الخروج إلا برمية 2 بالضبط
- **HORUS (المربع 30)**: الحجر يمكنه الخروج بأي رمية (1، 2، 3، 4، أو 5)

(b) قواعد الخروج من اللوح:

- إذا كان الهدف < 30 (خارج اللوح):
 - يُسمح بالخروج فقط للأحجار التي وصلت المربعات 26-30 (البيوت الخمسة الأخيرة)
 - الأحجار في المربعات 1-25 لا يمكنها القفز للخروج
 - لا يمكن للأحجار التي لم تصل المربع 26 أن تتخطاه .

(c) قواعد الحركة العادية:

- لا يمكن الحركة للخلف (`targetPos < 1`)
- إذا كان المربع الهدف فارغاً: حركة عادية مسموحة
- إذا كان المربع الهدف مشغولاً :
 - إذا كان مشغولاً بحجر من نفس اللاعب: الحركة ممنوعة
 - إذا كان مشغولاً بحجر الخصم: مبادلة (`swap`) تحدث

3. كائن **Move** إذا كانت الحركة ممكنة، أو **null** إذا كانت ممنوعة.

4. `applySpecialCellRules(Board board, Piece piece, int position, boolean silent)`: دالة خاصة (private) تطبق القواعد الفورية للمربعات الخاصة عندما يهبط عليها حجر.

القواعد المطبقة:

(a) **WATER (المربع 27)**:

- يعيد الحجر فوراً إلى بيت البعث (المربع 15 أو أقرب مربع فارغ قبله)
- عقوبة قاسية لأنها تُرجع الحجر للخلف بمسافة كبيرة

(b) **THREE_TRUTHS (المربع 28)** :

- يُحتجز الحجر ويُعلم بـ `needsExactRoll = 3`
- لا يمكنه الخروج إلا برمية 3 بالضبط في دوره القادم

c) RE_ATOUM (المربع 29) :

- يُحتجز الحجر ويُعلم بـ `needsExactRoll = 2`
- لا يمكنه الخروج إلا برمية 2 بالضبط

d) HORUS (المربع 30):

- يُعلم الحجر بـ `needsExactRoll = 1`
- يمكنه الخروج بأي رمية (1، 2، 3، 4، أو 5)

e) HAPPINESS (المربع 26) :

- مربع محايد، لا عقوبات ولا مكافآت ولكن يجب ان يمر عليه الحجر.
- مجرد طباعة رسالة

f) REBIRTH (المربع 15) :

- بيت البعث، مجرد طباعة رسالة عند الوصول إليه.

5. `penalizeSpecialHouses(Board board, PlayerType player, Move appliedMove, boolean silent)` .

هذه الدالة المسؤولة عن تطبيق عقوبات البيوت الخاصة على الأجار التي لم تتحرك منها .

6. `returnToRebirth(Board board, Piece piece)` : دالة مساعدة خاصة (private) تُعيد حجراً إلى بيت البعث.

7. `isValidMove(Board board, Move move)` : دالة للتحقق من صحة حركة معينة. تُستخدم بشكل أساسي للتحقق من مدخلات اللاعب البشري.

8. `cloneBoard(Board board)` : دالة لإنشاء نسخة عميقة (deep copy) من اللوح. تستخدم `copy constructor` الموجود في كلاس `Board` .

Expectiminimax في أهمية في ضرورة جداً لأننا لا نريد تعديل حالة اللعبة الأصلية عند استكشاف شجرة البحث. كل فرع في الشجرة يحتاج نسخة مستقلة من اللوح لتطبيق الحركات عليها.

9. `applyMoveCloned(Board board, Move move)` : دالة مركبة تنسخ اللوح، تطبق حركة على النسخة، وتُرجع اللوح الجديد. هذه الدالة تجمع بين `cloneBoard()` و `applyMove()` في خطوة واحدة.

10. `printValidMoves(List<Move> moves)` : دالة مساعدة لطباعة قائمة الحركات الممكنة بشكل منسق. تُستخدم في واجهة اللاعب البشري.

: class Cost.3

الوصف العام:

يمثل هذا الكلاس نظام حساب التكلفة في اللعبة. التكلفة هنا تمثل "المسافة" الذي حققته الأحجار في اللعبة.

الدوال:

1. `calculateTotalCost(Board board, PlayerType player)` : تحسب التكلفة الإجمالية لجميع أحجار لاعب معين. التكلفة الإجمالية هي مجموع تكاليف جميع الأحجار.
2. `calculatePieceCost(Piece piece)` : تحسب تكلفة حجر واحد بناءً على موقعه الحالي مقارنة بموقعه الابتدائي.
3. `getInitialPosition(Piece piece)` : دالة مساعدة خاصة (private) تُرجع الموقع الابتدائي لحجر بناءً على لونه ورقمه.
4. `calculateMoveCost(Move move)` : تحسب تكلفة الانتقال (transition cost) لحركة معينة.
5. `calculateCumulativeCost(GameState state, PlayerType player)` : تحسب التكلفة التراكمية لحالة معينة، آخذة بعين الاعتبار كلاً من تقدم الأحجار وعدد الأدوار.
6. `calculateAverageCost(Board board, PlayerType player)` : تحسب متوسط التكلفة لكل حجر للاعب معين.
7. `printCostDetails(GameState state, PlayerType player)` : دالة طباعة تفصيلية لجميع معلومات التكلفة الخاصة بلاعب معين.

: Class Board.4

الوصف العام:

يمثل هذا الكلاس لوحة اللعبة الكاملة (Game Board) التي تتكون من 30 مربعاً. هو المسؤول عن إدارة جميع الخلايا (Cells) وجميع الأحجار (Pieces) للاعبين، ويوفر واجهة شاملة للتفاعل مع حالة اللوحة.

Board هو الهيكل الأساسي الذي يحتوي على كل ما يتعلق بالرقعة المادية للعبة. يمكن اعتباره "النموذج الفيزيائي" للعبة الذي يحتفظ بموقع كل قطعة وحالة كل مربع. يُستخدم هذا الكلاس بشكل مكثف في:

- تطبيق قواعد اللعبة
- التحقق من صحة الحركات
- حساب دالة التقييم (Heuristic)
- نسخ حالات اللعبة للمحاكاة في خوارزمية Expectiminimax

المتغيرات الرئيسية:

cells: مصفوفة من 31 عنصراً من نوع Cell (نتجاهل الفهرس 0 ونستخدم 1-30 لتوافق مع أرقام المربعات الفعلية). كل عنصر يمثل مربعاً واحداً على اللوح.

whitePieces: قائمة (List<Piece>) تحتوي على جميع الأحجار البيضاء (7 أحجار) الخاصة باللاعب الكمبيوتر.

blackPieces: قائمة تحتوي على جميع الأحجار السوداء (7 أحجار) الخاصة باللاعب البشري.

البناءات: (Constructors)

الباني الأول: (Default Constructor)

ينشئ لوحة جديدة في حالتها الابتدائية:

- يُنشئ 30 خلية من 1 إلى 30
- يُنشئ 7 أحجار بيضاء و 7 أحجار سوداء
- يوزع الأحجار بالتناوب على المربعات الـ 14 الأولى (الأحجار البيضاء على المربعات الفردية 1,3,5... والأحجار السوداء على المربعات الزوجية 2,4,6...)
- يربط كل حجر بالخلية المناسبة

الباني الثاني: (Copy Constructor)

ينشئ نسخة عميقة (Deep Copy) من لوحة موجودة:

- ينسخ جميع الخلايا بشكل مستقل
 - ينسخ جميع الأحجار بشكل مستقل
 - يعيد ربط الأحجار المنسوخة بالخلايا المنسوخة
- أهمية النسخ العميق: ضروري جداً في خوارزمية Expectiminimax لأننا نحتاج لمحاكاة الحركات على نسخ من اللوحة دون التأثير على الحالة الأصلية.

الدوال الرئيسية:

دوال الوصول الأساسية:

getCell(int index): تُرجع الخلية عند موقع معين (1-30)، تُرجع null إذا كان الموقع خارج النطاق.

getPieceAt(int position): تُرجع الحجر الموجود على موقع معين، أو null إذا كان المربع فارغاً.

getPieces(PlayerType player): تُرجع نسخة من قائمة أحجار لاعب معين، مفيدة لحساب التقييم والحركات الممكنة.

isSquareOccupied(int position): يتحقق إذا كان مربع معين مشغولاً بحجر.

findPiece(int id, PlayerType owner): تبحث عن حجر معين بناءً على معرفه ومالكه في هذه اللوحة. مهمة جداً عند تطبيق حركة على لوحة منسوخة، لأننا نحتاج للعثور على الحجر المقابل في النسخة الجديدة.

دوال فحص انتهاء اللعبة:

isGameOver(): يتحقق إذا كانت اللعبة قد انتهت (أي لاعب أنهى جميع أحجاره).

isPlayerFinished(PlayerType player): يتحقق إذا كان لاعب معين قد أنهى جميع أحجاره.

getWinner(): تُرجع اللاعب الفائز، أو null إذا لم تنتهِ اللعبة بعد.

دوال الإحصائيات والتقييم:

countPiecesOnBoard(PlayerType player): تحسب عدد الأحجار التي لا تزال على اللوحة للاعب معين.

calculateProgress(PlayerType player): تحسب التقدم الإجمالي لأحجار لاعب معين:

- كل حجر منتهٍ يُعطى 35 نقطة
- كل حجر على اللوحة يُعطى نقاطاً بقيمة موقعه

هذه الدالة أساسية في دالة التقييم (Heuristic Function)

دوال الطباعة:

printBoard(): تطبع اللوحة بشكل مرئي جميل باستخدام رموز Unicode ، تُظهر:

- الصفوف الثلاثة بترتيبها الصحيح
- رموز الأحجار (●/○)
- رموز المربعات الخاصة (Ψ/😊/≈/⚓/♀/⊕)
- إطارات جميلة حول اللوحة

printStatistics(): تطبع إحصائيات اللعب الحالية:

- عدد الأحجار على اللوحة لكل لاعب
- عدد الأحجار المنتهية
- التقدم الإجمالي

printPiecesInfo(PlayerType player): تطبع معلومات تفصيلية عن أحجار لاعب معين.

الأهمية في المشروع:

1. التمثيل المركزي: كل شيء في اللعبة يرتبط بالـ Board الحركات تُطبق عليه، القواعد تُفحص من خلاله، والتقييم يُحسب منه.
2. النسخ للمحاكاة: يوفر نسخاً عميقة كاملة تسمح بمحاكاة آلاف الحركات دون التأثير على اللعبة الفعلية.

3. الفصل بين البيانات والمنطق :يحتفظ بالبيانات فقط، بينما القواعد موجودة في Rules.java، مما يحقق تصميماً نظيفاً.

5. Class GameState :

الوصف العام:

يمثل هذا الكلاس حالة اللعبة الكاملة في لحظة زمنية محددة. هو وحدة الحالة (State) في تمثيل المسألة كمسألة بحث، ويحتوي على كل المعلومات اللازمة لوصف اللعبة بشكل كامل في أي نقطة زمنية. في خوارزمية Expectiminimax، نتعامل مع شجرة من حالات اللعبة، حيث كل عقدة تمثل GameState معينة، والانتقال من عقدة لأخرى يتم عبر تطبيق Move أو رمي عصي (dice roll). GameState ليس مجرد وعاء للبيانات، بل هو الواجهة الموحدة التي تستخدمها الخوارزمية للتفاعل مع اللعبة.

المتغيرات الرئيسية:

board: كائن من نوع Board يمثل حالة اللوحة الحالية (مواقع الأحجار).
currentPlayer: اللاعب الذي يجب أن يلعب في هذه الحالة (PlayerType.WHITE أو PlayerType.BLACK).
diceValue: قيمة رمية العصي الحالية (1-5)، تكون null إذا لم يتم رمي العصي بعد في هذا الدور.
turnNumber: رقم الدور الحالي، يبدأ من 1 ويزداد بعد كل دور.

البناءات (Constructors):

الباني الأول:

ينشئ حالة لعبة جديدة:

- يأخذ لوحة جاهزة ولاعباً ابتدائياً
- يضع `diceValue` (null لم يُرم بعد)
- يبدأ العداد من الدور رقم 1

الباني الثاني: (Copy Constructor)

ينشئ نسخة عميقة من حالة لعبة موجودة:

- ينسخ اللوحة بالكامل (Deep copy)
- ينسخ جميع المعلومات الأخرى

الاستخدام الأساسي : عند محاكاة الحركات في الخوارزمية، ننسخ الحالة الحالية، نطبق الحركة على النسخة، ونقيّمها، دون أن نمس الحالة الأصلية.

الدوال الرئيسية:

دوال الوصول: (Getters & Setters)

جميع المتغيرات لها getters و setters بسيطة:

- `getBoard(), setBoard()`
- `getCurrentPlayer(), setCurrentPlayer()`
- `getDiceValue(), setDiceValue()`
- `getTurnNumber(), setTurnNumber()`

دوال فحص اللعبة:

isGameOver(): تفوّض الفحص للوحة. (`board.isGameOver()`)

getWinner(): تفوّض الحصول على الفائز للوحة.

دوال الانتقال بين الحالات:

changeTurn(): تنتقل للاعب التالي:

- تبديل اللاعب الحالي (`currentPlayer = currentPlayer.getOpponent()`)
- تُصفر قيمة العصي (`diceValue = null`)
- تزيد رقم الدور

clone(): تُنشئ نسخة عميقة من الحالة باستخدام الـ Copy Constructor.

تطبيق الحركات:

applyMove(Move move): تطبيق حركة على الحالة الحالية:

1. تجد القطعة المقابلة في لوحة هذه الحالة (مهم عند النسخ)

2. تُنشئ حركة جديدة بالقطعة الصحيحة

3. تستدعي `Rules.applyMove()` لتطبيق الحركة فعلياً

applyMove(Move move, boolean silent): نفس الوظيفة السابقة لكن مع التحكم في الطباعة:

- `silent = true:` لا تطبع رسائل (تُستخدم في المحاكاة)
- `silent = false:` تطبع رسائل تفصيلية (تُستخدم في اللعب الفعلي)

توليد الحركات:

getValidMoves(): تُرجع قائمة بجميع الحركات الصحيحة الممكنة للاعب الحالي بناءً على قيمة العصي الحالية. تفوض هذه المهمة لـ `Rules.getValidMoves()`.

دوال الطباعة:

printStats(): تطبع معلومات الحالة الحالية:

- رقم الدور
- اللاعب الحالي
- قيمة رمية العصي

toString(): تمثيل نصي مختصر للحالة.

الأهمية في المشروع:

1. وحدة الحالة في البحث: كل عقدة في شجرة Expectiminimax هي `GameState`، مما يجعل الخوارزمية تتعامل مع واجهة موحدة بسيطة.

2. الفصل بين البيانات والعمليات:

- `GameState` يحتفظ بالحالة فقط
- `Rules` يطبق القواعد
- `SenetAI` يبحث في الحالات

هذا التصميم يحقق **Single Responsibility Principle**.

3. النسخ الآمن: يضمن أن المحاكاة لا تؤثر على اللعبة الفعلية، وهو أمر حيوي في خوارزميات البحث في الألعاب.

4. التحكم في السياق: يوفر `silent mode` الذي يسمح بتشغيل آلاف الحركات دون طباعة، مع الاحتفاظ بالقدرة على الطباعة التفصيلية عند اللعب الفعلي.

العلاقة بين `Board` و `GameState`:

- `GameState` يحتوي على `Board` (تركيب (Composition) -
 - `Board` يمثل الجانب الفيزيائي (أين الأحجار؟)
 - `GameState` يضيف السياق الزمني (من يلعب؟ ما الرمية؟ أي دور؟)
- معاً، يشكلان التمثيل الكامل لحالة اللعبة المطلوب في تمثيل المسألة كمسألة بحث.

استخدامهما في الخوارزمية:

// في كل عقدة من شجرة البحث:

GameState currentState = ...;

// نسخ الحالة

GameState nextState = currentState.clone();

// تطبيق حركة

nextState.applyMove(someMove, true); // silent mode

// تقييم الحالة الجديدة

double value = heuristic(nextState);

// الحالة الأصلية لم تتأثر!

هذا النمط يتكرر آلاف المرات في الخوارزمية، وهو السبب في أهمية النسخ العميق الصحيح.

: Controller 4.3

:class Game Controller

الوصف العام:

يعد ملف GameController.java المسؤول الرئيسي عن إدارة سير اللعبة وتنسيق التفاعل بين مختلف مكونات النظام، مثل حالة اللعبة، اللوحة، القوانين، والذكاء الاصطناعي.

يقوم GameController بدور المنسق حيث:

- يحدد دور اللاعب الحالي.
- يدير الانتقال بين الأدوار.
- يطبق الحركات المختارة.

- يستدعي القوانين الخاصة باللعبة دون أن يحتويها داخليًا.

: Constructor

الباني وظيفته المعتادة ينشئ كائن من نوع الكلاس لكن عند إنشاء كائن من GameController يجب مراعاة أن:

- يتم إنشاء لوحة اللعب.(Board)
- تهيئة حالة اللعبة.(GameState)
- تحديد أن اللاعب البشري يبدأ أولاً.
- إنشاء كائن الذكاء الاصطناعي (SenetAI) مع عمق محدد.
- تهيئة كائن الإدخال.(Scanner)

هذا يضمن أن كل لعبة تُدار بحالة مستقلة تمامًا.

بدء اللعبة:

تعمل الدالة startGame كحلقة التحكم الرئيسية، حيث:

- تستمر اللعبة طالما لم تتحقق حالة النهاية.
- يتم عرض حالة اللعبة واللوحة في كل دور.
- يتم تحديد ما إذا كان الدور للاعب البشري أو للذكاء الاصطناعي.
- يتم تنفيذ الدور المناسب.

دور اللاعب البشري:

في الدالة: playHumanTurn

- يتم رمي العصي وتحديد قيمة الحركة من قبل المستخدم .
- حساب جميع الحركات الصحيحة الممكنة.
- في حال عدم وجود حركات صالحة، يتم تخطي الدور وتطبيق العقوبات الخاصة.
- يسمح للمستخدم باختيار حركة من قائمة الحركات المتاحة.
- يتم تطبيق الحركة وتحديث حالة اللعبة.

دور الذكاء الاصطناعي:

في الدالة: playAITurn

- يتم رمي العصي تلقائياً.

- يحلل الذكاء الاصطناعي الحالة الحالية للعبة.
- يختار أفضل حركة ممكنة بناءً على خوارزمية اتخاذ القرار.
- يتم تطبيق الحركة المختارة وتحديث الحالة.

الذكاء الاصطناعي لا يعدّل حالة اللعبة مباشرة، بل يقترح حركة فقط، بينما يتولى `GameController` تنفيذها، أي البرنامج يفصل بين التفكير والتنفيذ.

إنهاء اللعبة:

عند انتهاء اللعبة:

- يتم عرض اللوحة النهائية.
- تحديد اللاعب الفائز.
- عرض عدد الأدوار والنتيجة النهائية.
- إغلاق الموارد المستخدمة.

: Main 5.3

: Class Main

الوصف العام:

كما في كل البرامج والألعاب يمثل نقطة الدخول الرئيسية للبرنامج ، وهو المسؤول عن بدء تشغيل اللعبة وإدارة التفاعل الأولي مع المستخدم. لا يحتوي على منطق اللعبة أو القوانين بل يستدعيها من الكلاسات الأخرى، ويركّز على عرض القوائم، قراءة اختيارات المستخدم، وتوجيه سير التنفيذ نحو الأجزاء المناسبة من النظام.

الهدف الأساسي من هذا الملف هو:

- عرض القائمة الرئيسية للعبة.
- تمكين المستخدم من اختيار نمط اللعب ومستوى الصعوبة.
- إنشاء كائن التحكم باللعبة (`GameController`) وتمرير الإعدادات المناسبة له.
- إعادة تشغيل اللعبة أو إنهائها بناءً على رغبة المستخدم.

بهذا الشكل، يحقق الملف فصلاً واضحاً بين واجهة المستخدم النصية ومنطق اللعبة الفعلي.

الدالة `main`:

تقوم الدالة `main` بما يلي:

- إنشاء كائن `Scanner` لقراءة إدخلات المستخدم.
- عرض القائمة الرئيسية باستخدام `displayMainMenu`.
- قراءة اختيار المستخدم والتحقق من صحته.
- توجيه التنفيذ باستخدام `switch` لاختيار نمط اللعب المناسب.

تشمل أنماط اللعب:

- لعب عادي مع ذكاء اصطناعي بعمق متوسط.
- لعب مع إظهار تفاصيل تحليل الذكاء الاصطناعي.
- اختيار مستوى الصعوبة يدويًا.
- عرض جدول احتمالات رمي العصي، والذي يضيف جانبًا تحليليًا وتعليميًا للعبة.

اختيار مستوى الصعوبة :

تتيح الدالة `selectDifficulty` للمستخدم اختيار مستوى صعوبة اللعب مع الذكاء الاصطناعي. يتم تحويل مستوى الصعوبة المختار إلى قيمة عمق (`Depth`) تُستخدم لاحقًا في خوارزمية اتخاذ القرار الخاصة بالذكاء الاصطناعي، حيث يشير العمق إلى عدد الحركات المستقبلية التي يتم تحليلها.

كلما زاد العمق:

- زادت دقة قرارات الذكاء الاصطناعي.
- زاد الزمن الحسابي المطلوب.

التحقق من الإدخال:

تم تخصيص الدالة `getIntInput` للقيام بـ `validation` لضمان :

- إدخال قيم رقمية صحيحة فقط.
- منع انهيار البرنامج بسبب إدخال غير صالح.
- تحسين موثوقية البرنامج وتجربة المستخدم.

١. اسم العيلة (قبيلة) صفوة (ثالث)

2- تمثيل المبرور (توضيح أسهل وأسود)

3- قنصل الأعرجار (عجاري، بيت إجنه، بيت إعدده)
بيت ملاء، بيت، كفاقر، اللثلاث، بيت إعدده، يوم،
بيت عورس).

أي يجب، هذا كذا نوع مع، قسم الحجر

4. کتابہ قوائد الصغار

5- تابع ~~محل~~ حرکت الحرف (الف) بعد ()
در مصراع اول شکل حرف 2

ولكنه المسموح بعدد الأعماد التي ظهرت
تحريك الحجر + تأكيده شروط اللعبة
(أي هو يوجد في هذه الماكلة تغيير الماكلة بينهم)
إذا كان في مكانه فوعى الحجر غير الحجر الذي يجب
أن يحققه الشرط وعندما يصل الحجر إلى
بيت، سعادة يجب أن يمر عليه بها
كان عدد الحركات

6- تابع لتتميل صاب، لقيم، لاصطلاح

كل حبة للعصا بشكل خاص
(إما لتقتل كل عصا لها أو لتحمّل إرمية
الكبالة احماليّة).

7- تابع رسوم لكل الحوافية Expectiminimax

8 - طباعة (عدد، لغة، فترة تاريخ، نص)

من المص: آجرة، برنولية.

عدد، لبتا في 16 = 2⁴

نستخدم التوافيق (nCr) لحساب الحالات

الاصالة متناظرة

- * عدم إزالة الحجارة من اللوحة عند خلعها.
- * عدم العرقلة في بيت العبة (معالجة حالة المواد فضفاضة)
- * عدم اعتبار ضربة ال block من الواجب إلا ~~في~~ عند الوصول إلى the house of three

رسمي المعنى تجربة برنولي لأن كل عصابة لا وفاسين
عدد النتائج : $2^n = 2^4 = 16$

لتحسب عدد الحالات لكل رابعة نستخدم احتماليته من خلال القانون

$$C(n, k) = \frac{n!}{k!(n-k)!}$$

$$\Rightarrow C(4, k) = \frac{4!}{k!(4-k)!}$$

← العدد (0) : (تقابل 5 في اعتنا)

$$C(4, 0) = \frac{4!}{0!(4-0)!} = 1$$

* عدد مرات ظهوره في (0 مرة)
* احتمالية ظهوره هي $\frac{1}{16}$

$$\leftarrow \text{العدد (1)} : C(4, 1) = \frac{4!}{1!(4-1)!} = 4$$

* عدد مرات ظهوره هي (4 مرات)
* احتمالية ظهوره هي $\frac{4}{16}$

$$\leftarrow \text{العدد (2)} : C(4, 2) = \frac{4!}{2!(4-2)!} = \frac{4!}{2! \times 2!} = 6$$

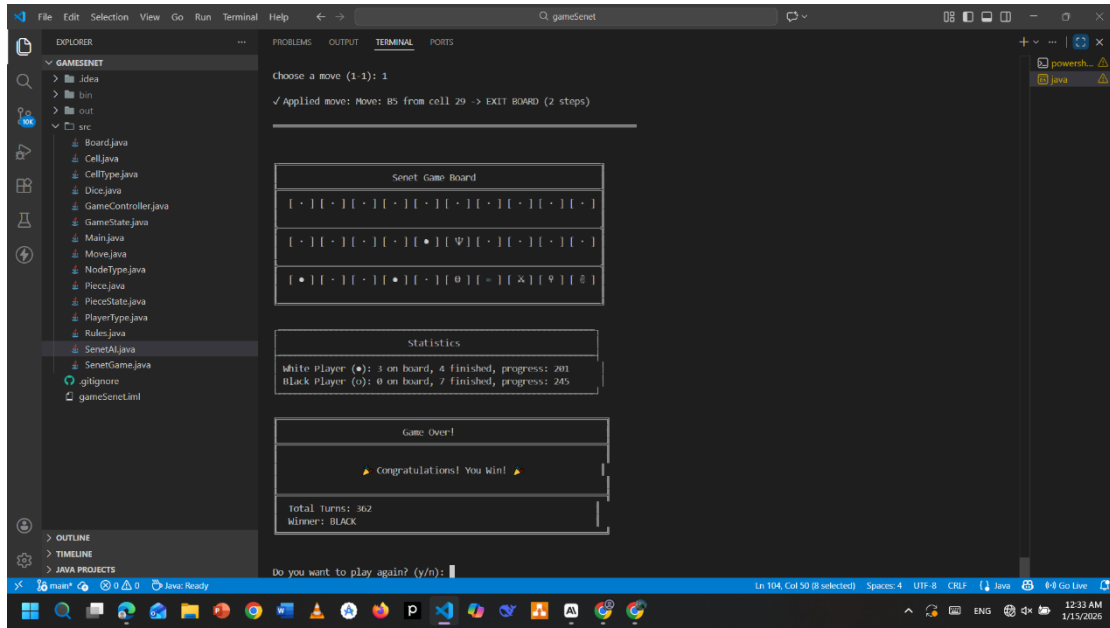
* عدد مرات ظهوره هي (6 مرات)
* احتمالية ظهوره هي $\frac{6}{16}$

$$\leftarrow \text{العدد (3)} : C(4, 3) = \frac{4!}{3!(4-3)!} = \frac{4!}{3! \times 1!} = 4$$

* عدد مرات ظهوره هي (4 مرات)
* احتمالية ظهوره هي $\frac{4}{16}$

$$\leftarrow \text{العدد (4)} : C(4, 4) = \frac{4!}{4! \times 0!} = 1$$

* عدد مرات ظهوره هي (مرة واحدة)
* احتمالية ظهوره هي $\frac{1}{16}$



تم تقسيم العمل وفق التالي :

اسم الطالب / الطالبة	أسماء الكلاسات
سدره الحمد	Cell, CellType , Move , NodeType
آية الحصري	Piece , PieceState , PlayerType
شام العلي	Dice, Rules, Cost ,SenetAI
حيدر إبراهيم	Board, GameState
جودي برادعي	Main, Game Controller